
Pyramid / Diamond Pattern Menu Program

Submitted by

Name	Rudra Pratap Singh
SAP ID	590032811
Batch	16

1. Problem Statement

Write a C program that displays a menu and uses a switch statement to print the Pyramid / Diamond pattern chosen by the user. Only nested loops may be used (no arrays), and each pattern is implemented in a separate parameterless function that returns no value. For every activity the algorithm and a test-case table are prepared first; then the program is written, compiled, executed and tested against those test cases.

The five patterns required are:

No.	Pattern	Rule
1	Full pyramid	Row i has $(2i - 1)$ asterisks
2	Inverted pyramid	Row 1 has $(2n - 1)$ asterisks, reducing by 2 each row
3	Diamond pattern	$(2n - 1)$ rows; the middle row has $(2n - 1)$ asterisks
4	Palindromic number pyramid	Row i : 1 to i , then back down to 1
5	Repeated-number pyramid	Row i : the number i printed $(2i - 1)$ times

2. Program Design

2.1 Overall Approach

The `main()` function shows the menu inside a do-while loop, reads the user's choice and passes it to a switch statement. Each case calls one pattern function, so the menu keeps coming back until the user selects 6 (Exit). Each pattern function reads its own value of n , checks that it is valid, and then prints the pattern with nested for loops. No arrays are used anywhere in the program.

2.2 Functions Used

Function	Purpose
<code>main()</code>	Displays the menu, reads the choice, and calls the required function through switch
<code>fullPyramid()</code>	Prints the full pyramid (Activity 1)
<code>invertedPyramid()</code>	Prints the inverted pyramid (Activity 2)
<code>diamondPattern()</code>	Prints the diamond pattern (Activity 3)
<code>palindromicPyramid()</code>	Prints the palindromic number pyramid (Activity 4)
<code>repeatedNumberPyramid()</code>	Prints the repeated-number pyramid (Activity 5)

2.3 Design Decisions

Patterns are centred by printing $2 \times (n - i)$ leading spaces before row i . This is because every symbol is followed by a space, so each symbol takes two character positions.

The number of rows is restricted to 1-9. This keeps all numbers single-digit so the number pyramids stay aligned. The same limit is used for all five patterns for consistency.

If the user types something that is not a number (for example a letter), the bad input is discarded and the program shows an error message instead of going into an infinite loop.

2.4 Algorithm for main() - Menu

1. Start.
2. Display the menu with options 1 to 6.
3. Read the choice. If the input is not an integer, discard it and set choice = 0.
4. Use switch (choice):
 - a) case 1: call fullPyramid()
 - b) case 2: call invertedPyramid()
 - c) case 3: call diamondPattern()
 - d) case 4: call palindromicPyramid()
 - e) case 5: call repeatedNumberPyramid()
 - f) case 6: print the exit message
 - g) default: print "Invalid choice"
5. If choice is not 6, go back to step 2.
6. Stop.

3. Activity 1: Full Pyramid

Aim

Read the number of rows n and print a centred full pyramid of asterisks. Row i contains $(2i - 1)$ asterisks.

Sample pattern ($n = 4$)

```

      *
     * * *
    * * * * *
   * * * * * * *
```

Algorithm - fullPyramid()

1. Start the function fullPyramid().
2. Read n . If the input is not an integer, or $n < 1$, or $n > 9$, print an error message and return to the menu.
3. Repeat for $i = 1$ to n (one pass per row):
 - a) Print $2 \times (n - i)$ spaces using an inner loop ($j = 1$ to $2(n - i)$).
 - b) Print $(2i - 1)$ asterisks, each followed by a space, using an inner loop.
 - c) Print a newline.
4. End of function; control returns to main().

Test-Case Table

Menu choice for this activity: 1

Test ID	Input (n)	Expected Output	Actual Output	Result
T1.1	1	1 row: 1 star	1 row: 1 star	Pass
T1.2	3	3 rows: 1, 3, 5 stars	3 rows: 1, 3, 5 stars	Pass
T1.3	4	4 rows: 1, 3, 5, 7 stars	4 rows: 1, 3, 5, 7 stars	Pass
T1.4	0 (invalid)	Error message; no pattern printed	Error message shown; returned to menu	Pass
T1.5	x (non-numeric)	Error message; no pattern printed	Error message shown; returned to menu	Pass

4. Activity 2: Inverted Pyramid

Aim

Read the number of rows n and print an inverted pyramid. The first row has $(2n - 1)$ asterisks and each following row has two fewer.

Sample pattern ($n = 4$)

```
* * * * *
 * * * *
  * * *
   * *
```

Algorithm - invertedPyramid()

1. Start the function invertedPyramid().
2. Read n . If the input is not an integer, or $n < 1$, or $n > 9$, print an error message and return to the menu.
3. Repeat for $i = n$ down to 1 (one pass per row):
 - a) Print $2 \times (n - i)$ spaces using an inner loop.
 - b) Print $(2i - 1)$ asterisks, each followed by a space, using an inner loop.
 - c) Print a newline.
4. End of function; control returns to main().

Test-Case Table

Menu choice for this activity: 2

Test ID	Input (n)	Expected Output	Actual Output	Result
T2.1	1	1 row: 1 star	1 row: 1 star	Pass
T2.2	3	3 rows: 5, 3, 1 stars	3 rows: 5, 3, 1 stars	Pass
T2.3	4	4 rows: 7, 5, 3, 1 stars	4 rows: 7, 5, 3, 1 stars	Pass
T2.4	0 (invalid)	Error message; no pattern printed	Error message shown; returned to menu	Pass
T2.5	x (non-numeric)	Error message; no pattern printed	Error message shown; returned to menu	Pass

5. Activity 3: Diamond Pattern

Aim

Read n , the number of rows in the upper half, and print a diamond. The middle row has $(2n - 1)$ asterisks and the complete diamond has $(2n - 1)$ rows.

Sample pattern ($n = 4$)

```

      *
     * * *
    * * * * *
   * * * * * *
  * * * * * *
 * * * * *
  * * *
   * *
    *

```

Algorithm - diamondPattern()

1. Start the function diamondPattern().
2. Read n . If the input is not an integer, or $n < 1$, or $n > 9$, print an error message and return to the menu.
3. Upper half: repeat for $i = 1$ to n :
 - a) Print $2 \times (n - i)$ spaces.
 - b) Print $(2i - 1)$ asterisks, each followed by a space.
 - c) Print a newline.
4. Lower half: repeat for $i = n - 1$ down to 1:
 - a) Print $2 \times (n - i)$ spaces.
 - b) Print $(2i - 1)$ asterisks, each followed by a space.
 - c) Print a newline.
5. End of function; control returns to main().

Test-Case Table

Menu choice for this activity: 3

Test ID	Input (n)	Expected Output	Actual Output	Result
T3.1	1	1 row: 1 star	1 row: 1 star	Pass
T3.2	3	5 rows: 1, 3, 5, 3, 1 stars	5 rows: 1, 3, 5, 3, 1 stars	Pass
T3.3	4	7 rows: 1, 3, 5, 7, 5, 3, 1 stars	7 rows: 1, 3, 5, 7, 5, 3, 1 stars	Pass
T3.4	0 (invalid)	Error message; no pattern printed	Error message shown; returned to menu	Pass
T3.5	x (non-numeric)	Error message; no pattern printed	Error message shown; returned to menu	Pass

6. Activity 4: Palindromic Number Pyramid

Aim

Read n and print a pyramid in which row i shows the numbers 1 to i in increasing order and then $i - 1$ back down to 1.

Sample pattern ($n = 4$)

```

      1
     1 2 1
    1 2 3 2 1
   1 2 3 4 3 2 1

```

Algorithm - palindromicPyramid()

1. Start the function palindromicPyramid().
2. Read n . If the input is not an integer, or $n < 1$, or $n > 9$, print an error message and return to the menu.
3. Repeat for $i = 1$ to n :
 - a) Print $2 \times (n - i)$ spaces.
 - b) Increasing part: for $j = 1$ to i , print j followed by a space.
 - c) Decreasing part: for $j = i - 1$ down to 1, print j followed by a space.
 - d) Print a newline.
4. End of function; control returns to main().

Test-Case Table

Menu choice for this activity: 4

Test ID	Input (n)	Expected Output	Actual Output	Result
T4.1	1	1	1	Pass
T4.2	3	1 / 1 2 1 / 1 2 3 2 1	1 / 1 2 1 / 1 2 3 2 1	Pass
T4.3	4	1 / 1 2 1 / 1 2 3 2 1 / 1 2 3 4 3 2 1	1 / 1 2 1 / 1 2 3 2 1 / 1 2 3 4 3 2 1	Pass
T4.4	0 (invalid)	Error message; no pattern printed	Error message shown; returned to menu	Pass
T4.5	x (non-numeric)	Error message; no pattern printed	Error message shown; returned to menu	Pass

7. Activity 5: Repeated-Number Pyramid

Aim

Read n and print a pyramid in which row i contains the number i printed $(2i - 1)$ times.

Sample pattern ($n = 4$)

```

    1
  2 2 2
3 3 3 3 3
4 4 4 4 4 4 4
```

Algorithm - repeatedNumberPyramid()

1. Start the function repeatedNumberPyramid().
2. Read n . If the input is not an integer, or $n < 1$, or $n > 9$, print an error message and return to the menu.
3. Repeat for $i = 1$ to n :
 - a) Print $2 \times (n - i)$ spaces.
 - b) For $j = 1$ to $(2i - 1)$, print the value i followed by a space.
 - c) Print a newline.
4. End of function; control returns to main().

Test-Case Table

Menu choice for this activity: 5

Test ID	Input (n)	Expected Output	Actual Output	Result
T5.1	1	1	1	Pass
T5.2	3	1 / 2 2 2 / 3 3 3 3 3	1 / 2 2 2 / 3 3 3 3 3	Pass
T5.3	4	1 / 2 2 2 / 3 3 3 3 3 / 4 4 4 4 4 4 4	1 / 2 2 2 / 3 3 3 3 3 / 4 4 4 4 4 4 4	Pass
T5.4	0 (invalid)	Error message; no pattern printed	Error message shown; returned to menu	Pass
T5.5	x (non-numeric)	Error message; no pattern printed	Error message shown; returned to menu	Pass

8. Test Cases for the Menu

The following test cases check that the switch statement handles wrong menu choices and that the program exits correctly.

Test ID	Menu Input	Expected Output	Actual Output / Result
M1	7 (out of range)	Message "Invalid choice!", menu shown again	As expected - Pass
M2	abc (non-numeric)	Message "Invalid choice!", menu shown again, no infinite loop	As expected - Pass
M3	0	Message "Invalid choice!", menu shown again	As expected - Pass
M4	6 (exit)	Message "Exiting program", program ends	As expected - Pass

9. C Program (Source Code)

File name: *patterns.c*

```

/*
 * Program : Pyramid / Diamond Pattern Menu
 * Name    : Rudra Pratap Singh
 * SAP ID  : 590032811
 * Batch   : 16
 *
 * Description:
 *   Displays a menu and prints the pattern chosen by the user
 *   using a switch statement and nested loops (no arrays).
 *   Every pattern is written in its own parameterless function
 *   that returns nothing.
 */

#include <stdio.h>

/* Function prototypes */
void fullPyramid(void);
void invertedPyramid(void);
void diamondPattern(void);
void palindromicPyramid(void);
void repeatedNumberPyramid(void);

int main(void)
{
    int choice;

    do
    {
        printf("\n===== PATTERN MENU =====\n");
        printf("1. Full pyramid\n");
        printf("2. Inverted pyramid\n");
        printf("3. Diamond pattern\n");
        printf("4. Palindromic number pyramid\n");
        printf("5. Repeated-number pyramid\n");
        printf("6. Exit\n");
        printf("=====\n");
        printf("Enter your choice (1-6): ");

        if (scanf("%d", &choice) != 1)
        {
            /* Non-numeric input: discard the bad line and ask again */
            while (getchar() != '\n')
                ;
            choice = 0;
        }

        switch (choice)
        {
            case 1: fullPyramid();          break;
            case 2: invertedPyramid();      break;
            case 3: diamondPattern();       break;
            case 4: palindromicPyramid();   break;
            case 5: repeatedNumberPyramid(); break;
            case 6: printf("\nExiting program. Thank you!\n"); break;
            default: printf("\nInvalid choice! Please enter a number from 1 to
6.\n");
        }
        } while (choice != 6);

    return 0;
}

```

```

/* 1. Full pyramid: row i has (2i - 1) stars */
void fullPyramid(void)
{
    int n, i, j;

    printf("Enter the number of rows (1-9): ");
    if (scanf("%d", &n) != 1 || n < 1 || n > 9)
    {
        while (getchar() != '\n')
            ;
        printf("Invalid input! Rows must be between 1 and 9.\n");
        return;
    }

    printf("\n");
    for (i = 1; i <= n; i++)
    {
        for (j = 1; j <= 2 * (n - i); j++)          /* leading spaces */
            printf(" ");
        for (j = 1; j <= 2 * i - 1; j++)            /* stars */
            printf("* ");
        printf("\n");
    }
}

/* 2. Inverted pyramid: row i has (2(n - i + 1) - 1) stars */
void invertedPyramid(void)
{
    int n, i, j;

    printf("Enter the number of rows (1-9): ");
    if (scanf("%d", &n) != 1 || n < 1 || n > 9)
    {
        while (getchar() != '\n')
            ;
        printf("Invalid input! Rows must be between 1 and 9.\n");
        return;
    }

    printf("\n");
    for (i = n; i >= 1; i--)
    {
        for (j = 1; j <= 2 * (n - i); j++)          /* leading spaces */
            printf(" ");
        for (j = 1; j <= 2 * i - 1; j++)            /* stars */
            printf("* ");
        printf("\n");
    }
}

/* 3. Diamond: upper half of n rows, then lower half of (n - 1) rows */
void diamondPattern(void)
{
    int n, i, j;

    printf("Enter the number of rows for the upper half (1-9): ");
    if (scanf("%d", &n) != 1 || n < 1 || n > 9)
    {
        while (getchar() != '\n')
            ;
        printf("Invalid input! Rows must be between 1 and 9.\n");
        return;
    }
}

```

```

printf("\n");
/* Upper half (including the middle row) */
for (i = 1; i <= n; i++)
{
    for (j = 1; j <= 2 * (n - i); j++)
        printf(" ");
    for (j = 1; j <= 2 * i - 1; j++)
        printf("* ");
    printf("\n");
}
/* Lower half */
for (i = n - 1; i >= 1; i--)
{
    for (j = 1; j <= 2 * (n - i); j++)
        printf(" ");
    for (j = 1; j <= 2 * i - 1; j++)
        printf("* ");
    printf("\n");
}
}

/* 4. Palindromic number pyramid: 1 2 ... i ... 2 1 in row i */
void palindromicPyramid(void)
{
    int n, i, j;

    printf("Enter the number of rows (1-9): ");
    if (scanf("%d", &n) != 1 || n < 1 || n > 9)
    {
        while (getchar() != '\n')
            ;
        printf("Invalid input! Rows must be between 1 and 9.\n");
        return;
    }

    printf("\n");
    for (i = 1; i <= n; i++)
    {
        for (j = 1; j <= 2 * (n - i); j++) /* leading spaces */
            printf(" ");
        for (j = 1; j <= i; j++) /* increasing part */
            printf("%d ", j);
        for (j = i - 1; j >= 1; j--) /* decreasing part */
            printf("%d ", j);
        printf("\n");
    }
}

/* 5. Repeated-number pyramid: row number printed (2i - 1) times */
void repeatedNumberPyramid(void)
{
    int n, i, j;

    printf("Enter the number of rows (1-9): ");
    if (scanf("%d", &n) != 1 || n < 1 || n > 9)
    {
        while (getchar() != '\n')
            ;
        printf("Invalid input! Rows must be between 1 and 9.\n");
        return;
    }

    printf("\n");
    for (i = 1; i <= n; i++)
    {
        for (j = 1; j <= 2 * (n - i); j++) /* leading spaces */
            printf(" ");

```

```
        for (j = 1; j <= 2 * i - 1; j++)          /* repeated number */
            printf("%d ", i);
        printf("\n");
    }
}
```

10. Compilation and Execution

Compilation

The program was compiled with GCC using the warning options -Wall and -Wextra:

```
$ gcc -Wall -Wextra -o patterns patterns.c
(no errors and no warnings - compiled successfully)
$ ./patterns
```

Menu Screen

```
===== PATTERN MENU =====
1. Full pyramid
2. Inverted pyramid
3. Diamond pattern
4. Palindromic number pyramid
5. Repeated-number pyramid
6. Exit
=====
Enter your choice (1-6):
```

Activity 1: Full Pyramid

Test T1.3 (n = 4)

```
Enter your choice (1-6): 1
Enter the number of rows (1-9): 4

  *
 * * *
* * * * *
* * * * * * *
```

Test T1.1 (n = 1)

```
Enter your choice (1-6): 1
Enter the number of rows (1-9): 1

*
```

Test T1.4 (n = 0, invalid)

```
Enter your choice (1-6): 1
Enter the number of rows (1-9): 0
Invalid input! Rows must be between 1 and 9.
```

Activity 2: Inverted Pyramid

Test T2.3 (n = 4)

```
Enter your choice (1-6): 2
Enter the number of rows (1-9): 4

* * * * *
 * * * *
  * * *
   * *
    *
```

Test T2.1 (n = 1)

```
Enter your choice (1-6): 2
```

```
Enter the number of rows (1-9): 1
```

```
*
```

Test T2.4 (n = 0, invalid)

```
Enter your choice (1-6): 2
```

```
Enter the number of rows (1-9): 0
```

```
Invalid input! Rows must be between 1 and 9.
```

Activity 3: Diamond Pattern

Test T3.3 (n = 4)

```
Enter your choice (1-6): 3
```

```
Enter the number of rows for the upper half (1-9): 4
```

```

      *
    * * *
  * * * * *
* * * * * * *
  * * * * *
    * * *
      *
    
```

Test T3.1 (n = 1)

```
Enter your choice (1-6): 3
```

```
Enter the number of rows for the upper half (1-9): 1
```

```
*
```

Test T3.4 (n = 0, invalid)

```
Enter your choice (1-6): 3
```

```
Enter the number of rows for the upper half (1-9): 0
```

```
Invalid input! Rows must be between 1 and 9.
```

Activity 4: Palindromic Number Pyramid

Test T4.3 (n = 4)

```
Enter your choice (1-6): 4
```

```
Enter the number of rows (1-9): 4
```

```

      1
    1 2 1
  1 2 3 2 1
1 2 3 4 3 2 1
    
```

Test T4.1 (n = 1)

```
Enter your choice (1-6): 4
```

```
Enter the number of rows (1-9): 1
```

```
1
```

Test T4.4 (n = 0, invalid)

```
Enter your choice (1-6): 4
```

```
Enter the number of rows (1-9): 0
```

```
Invalid input! Rows must be between 1 and 9.
```

Activity 5: Repeated-Number Pyramid

Test T5.3 (n = 4)

```
Enter your choice (1-6): 5
Enter the number of rows (1-9): 4
```

```
  1
 2 2 2
3 3 3 3 3
4 4 4 4 4 4 4
```

Test T5.1 (n = 1)

```
Enter your choice (1-6): 5
Enter the number of rows (1-9): 1
```

```
1
```

Test T5.4 (n = 0, invalid)

```
Enter your choice (1-6): 5
Enter the number of rows (1-9): 0
Invalid input! Rows must be between 1 and 9.
```

Wrong Menu Choice and Exit

```
Enter your choice (1-6): 7

Invalid choice! Please enter a number from 1 to 6.

(menu is shown again)
Enter your choice (1-6): 6

Exiting program. Thank you!
```

11. Test Summary

Group	Test Cases	Passed
Activity 1 - Full pyramid	5	5
Activity 2 - Inverted pyramid	5	5
Activity 3 - Diamond pattern	5	5
Activity 4 - Palindromic number pyramid	5	5
Activity 5 - Repeated-number pyramid	5	5
Menu tests (M1 - M4)	4	4
Total	29	29

12. Conclusion

The program displays a menu and prints all five patterns (full pyramid, inverted pyramid, diamond, palindromic number pyramid and repeated-number pyramid) using a switch statement and nested loops without arrays. Every pattern is written in its own parameterless function with no return value.

The patterns were tested with the prepared test cases, including the smallest input ($n = 1$), normal inputs ($n = 3$ and $n = 4$) and invalid inputs (0 and a non-numeric character). All 29 test cases produced the expected output. The exercise helped in understanding how the number of spaces and symbols in each row depends on the loop variable, and how nested loops build a two-dimensional pattern row by row.